



Razvoj, pokretanje i debugiranje u Visual Studio Code-u

Detaljne upute korak po korak za Windows 11

VS Code	C/C++	G++	GDB
---------	-------	-----	-----

Što ćete nakon ovih uputa moći? Instalirati sve potrebne alate, napisati prvi C++ program, kompajlirati ga, pokrenuti iz VS Code-a, postaviti breakpoint i pratiti izvršavanje programa korak po korak.

Namijenjeno početnicima, učenicima i nastavnicima

Ažurirano i tehnički provjereno: 28. rujna 2026.

Sadržaj i cilj vodiča

Ovaj vodič pretpostavlja da na računalu imate Windows 11 i da još nemate pripremljeno C++ razvojno okruženje. Svaki korak navodi što otvoriti, što preuzeti, što kliknuti, što upisati i kako provjeriti da je korak uspješno završen.

Važno Visual Studio Code je uređivač koda. Microsoftova C/C++ ekstenzija dodaje IntelliSense i integraciju s alatima, ali ne sadrži compiler ni debugger. Zato dodatno instaliramo GCC/G++ i GDB kroz MSYS2.

1	Što instaliramo i zašto
2	Instalacija Visual Studio Code-a
3	Instalacija Microsoft C/C++ ekstenzije
4	Instalacija MSYS2
5	Instalacija GCC/G++ i GDB alata
6	Dodavanje UCRT64 alata u PATH
7	Provjera instalacije
8	Povezivanje IntelliSense-a s pravim compilerom
9	Izrada prvog C++ projekta
10	Ručno kompajliranje i pokretanje
11	Pokretanje iz VS Code-a
12	Debugiranje programa
13	Praćenje varijabli i korak-po-korak izvršavanje
14	Preporučene compiler opcije
15	Najčešći problemi i rješenja
16	Sažetak i brzi podsjetnik
17	Službeni izvori i poveznice

1. Što instaliramo i zašto

Za rad u C++ jeziku koristit ćemo sljedeći razvojni sklop:

Komponenta	Uloga	Što dobivamo
Visual Studio Code	uređivač koda	pisanje i organizacija C++ datoteka
C/C++ ekstenzija	integracija u VS Code	IntelliSense, označavanje grešaka, build/debug integracija
GCC / G++	compiler	pretvara .cpp izvorni kod u .exe program
GDB	debugger	breakpointi, korak-po-korak izvršavanje i pregled varijabli
MSYS2 UCRT64	distribucija alata	jednostavna instalacija i održavanje GCC/GDB alata na Windowsu

Zašto UCRT64? MSYS2 dokumentacija preporučuje UCRT64 kao zadani izbor kada korisnik nije siguran koji environment odabrati. UCRT64 koristi GCC, 64-bitnu arhitekturu i moderni Universal C Runtime.

2. Instalacija Visual Studio Code-a

2.1. Otvorite službenu stranicu za preuzimanje

Otvorite: <https://code.visualstudio.com/Download>

Prije preuzimanja provjerite arhitekturu računala: Settings → System → About → System type. Ako piše “64-bit operating system, x64-based processor”, koristite Windows x64 verziju VS Code-a. Za ARM64 računala treba odabrati ARM64 izdanje.

KONTROLNA TOČKA

Za ovaj vodič pretpostavlja se standardno Windows 11 x64 računalo. Ako je računalo ARM64, nemojte slijepo preuzimati x86_64 MSYS2 paket; koristite odgovarajuću ARM64 varijantu.

2.2. Pokrenite instalacijsku datoteku

Pokrenite preuzetu datoteku oblika:

```
VSCodeUserSetup-x64-...exe
```

U instalacijskom čarobnjaku prihvatite licencu i nastavite s Next. Na ekranu “Select Additional Tasks” preporučuje se uključiti:

- ☒ Add "Open with Code" action to Windows Explorer file context menu
- ☒ Add "Open with Code" action to Windows Explorer directory context menu
- ☒ Register Code as an editor for supported file types
- ☒ Add to PATH

Provjera Nakon završetka pokrenite Visual Studio Code. Ako se program normalno otvori, prvi korak je završen.

3. Instalacija Microsoft C/C++ ekstenzije

3.1. Otvorite Extensions

U lijevom stupcu VS Code-a kliknite ikonu Extensions ili pritisnite:

Ctrl + Shift + X

3.2. Pronađite službenu Microsoftovu ekstenziju

U tražilicu upišite “C++”. Pronađite ekstenziju:

C/C++
Publisher: Microsoft

Službena stranica ekstenzije: [Visual Studio Marketplace – C/C++](#)

Kliknite Install.

Što ova ekstenzija daje? Sintaksno označavanje, IntelliSense, automatsko dovršavanje, provjeru pogrešaka i integraciju s compilerom/debuggerom. Compiler i debugger i dalje moramo instalirati zasebno.

4. Instalacija MSYS2

4.1. Otvorite službenu stranicu MSYS2

Otvorite: <https://www.msys2.org/>

Na početnoj stranici nalazi se odjeljak Installation. Ako je prethodna provjera pokazala x64 procesor, odaberite GUI installer x86_64. Ako imate ARM64 uređaj, odaberite odgovarajući ARM64 installer.

Napomena Naziv instalacijske datoteke sadrži datum izdanja, pa će se broj u nazivu s vremenom mijenjati. Primjer: msys2-x86_64-YYYYMMDD.exe.

4.2. Pokrenite MSYS2 installer

Kada installer zatraži Installation Folder, preporučujem ostaviti zadanu lokaciju:

```
C:\msys64
```

Nastavite s Next → Install. Na kraju ostavite uključenu opciju za pokretanje MSYS2 i kliknite Finish.

4.3. Provjerite da je otvoren UCRT64 terminal

Nakon instalacije trebao bi se otvoriti terminal za UCRT64 environment. Ako se ne otvori, u Windows Start izborniku potražite “MSYS2 UCRT64” i pokrenite ga.

AKO MSYS2 VEĆ POSTOJI NA RAČUNALU

Ako ne radite svježu instalaciju nego na računalu već postoji starija MSYS2 instalacija, prije instalacije toolchaina ažurirajte pakete naredbom `pacman -Syu`. Ako MSYS2 zatraži zatvaranje terminala nakon ažuriranja runtimea, zatvorite ga, ponovno otvorite “MSYS2 UCRT64” i ponovno pokrenite `pacman -Syu` dok ne bude potpuno ažuriran.

Važno Za ove upute koristimo UCRT64, ne obični MSYS terminal. Microsoftov vodič za C++ u VS Code-u koristi isti UCRT64 toolchain.

5. Instalacija GCC/G++ i GDB alata

5.1. U MSYS2 UCRT64 terminal upišite naredbu

```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```

Pritisnite Enter. MSYS2 će prikazati popis paketa iz toolchain grupe.

5.2. Prihvatite zadani izbor paketa

Kada se pojavi pitanje slično:

```
Enter a selection (default=all):
```

nemojte ništa upisivati — samo pritisnite Enter. Time prihvaćate sve pakete iz grupe.

5.3. Potvrdite instalaciju

Kada se pojavi:

```
Proceed with installation? [Y/n]
```

upišite Y i pritisnite Enter.

Rezultat Bit će instalirani kompileri i razvojni alati, uključujući gcc, g++, gdb, make i binutils.

6. Dodavanje UCRT64 alata u Windows PATH

Da bi VS Code i obični Windows terminal mogli pronaći g++ i gdb, u korisničku varijablu PATH dodajemo mapu u kojoj se alati nalaze.

6.1. Otvorite postavke Environment Variables

1. Pritisnite Windows tipku.
2. Upišite: environment variables.
3. Odaberite: Edit environment variables for your account.

6.2. Uredite korisnički Path

4. U dijelu User variables označite Path.
5. Kliknite Edit.
6. Kliknite New.
7. Dodajte sljedeću putanju:

```
C:\msys64\ucrt64\bin
```

Potvrdite sve prozore s OK.

Važno Zatvorite sve stare terminale i ponovno pokrenite VS Code. Već otvoreni programi ne moraju vidjeti novododanu PATH vrijednost.

7. Provjera instalacije

7.1. Otvorite novi terminal u VS Code-u

Odaberite Terminal → New Terminal. U njemu redom pokrenite:

```
gcc --version
g++ --version
gdb --version
```

Za svaku naredbu mora se prikazati verzija alata. Ako Windows javlja da naredba nije pronađena, najčešći uzrok je pogrešan PATH ili činjenica da terminal nije ponovno otvoren nakon promjene PATH-a.

7.2. Provjerite stvarnu lokaciju programa

```
where.exe g++
where.exe gdb
```

Očekivani rezultat mora pokazivati programe iz mape C:\msys64\ucrt64\bin. Ako where.exe prikaže više rezultata, prvi rezultat treba biti UCRT64 verzija iz te mape; u suprotnom drugi MinGW/GCC iz PATH-a može preuzeti prednost.

```
C:\msys64\ucrt64\bin
```

Kontrolna točka Ako g++ --version i gdb --version rade u novom VS Code terminalu, razvojni alati su ispravno povezani s Windowsom i VS Code-om.

8. Povezivanje IntelliSense-a s pravim compilerom

Ovaj korak preporučujemo napraviti obavezno, osobito ako je na računalu već bio instaliran Visual Studio, Qt, Dev-C++, WSL ili neki drugi GCC/MinGW. Time VS Code-u izričito kažemo koji compiler treba koristiti za IntelliSense.

8.1. Odaberite IntelliSense konfiguraciju

```
Ctrl + Shift + P
C/C++: Select IntelliSense Configuration
C:\msys64\ucrt64\bin\g++.exe
```

Ako se navedena naredba ne pojavi ili želite ručno provjeriti postavku, pokrenite “C/C++: Edit Configurations (UI)” i u polju Compiler path postavite C:\msys64\ucrt64\bin\g++.exe.

KONTROLNA TOČKA

Nakon ove postavke standardni include poput #include <iostream> ne bi smio biti crveno podcrtan zbog nepoznate putanje do standardne biblioteke.

9. Izrada prvog C++ projekta

9.1. Napravite mapu projekta

Primjer:

```
C:\Cplusplus\PrviProgram
```

9.2. Otvorite mapu u VS Code-u

U VS Code-u odaberite File → Open Folder... i otvorite mapu PrviProgram. Ako se pojavi pitanje “Do you trust the authors of the files in this folder?”, za vlastiti projekt odaberite Yes, I trust the authors.

9.3. Kreirajte datoteku main.cpp

U Explorer panelu odaberite New File i nazovite datoteku:

```
main.cpp
```

Unesite sljedeći program:

```
#include <iostream>

using namespace std;

int main()
{
    cout << "Pozdrav iz C++ programa!" << endl;

    return 0;
}
```

Spremite datoteku tipkama Ctrl + S.

10. Ručno kompajliranje i pokretanje

Prije automatizacije kroz VS Code korisno je jednom ručno vidjeti što se stvarno događa.

10.1. Kompajlirajte program

U terminalu, dok se nalazite u mapi projekta, upišite:

```
g++ main.cpp -o main.exe
```

Značenje:

Dio naredbe	Značenje
g++	pokreće C++ compiler
main.cpp	izvorna C++ datoteka
-o main.exe	naziv izlazne izvršne datoteke

10.2. Pokrenite program

```
.\main.exe
```

Očekivani rezultat:

```
Pozdrav iz C++ programa!
```

11. Pokretanje iz Visual Studio Code-a

Nakon što smo dokazali da compiler radi, program možemo pokretati iz samog VS Code-a.

11.1. Otvorite main.cpp

U gornjem desnom dijelu editora pronađite gumb Run/Play (▶) ili koristite Run and Debug prikaz.

11.2. Odaberite C++ build/run opciju

Pri prvom pokretanju VS Code može ponuditi izbor compilera. Odaberite "C/C++: g++.exe build and run active file". Ako je ponuđeno više g++.exe instalacija, odaberite onu čija je putanja C:\msys64\ucrt64\bin\g++.exe.

Što VS Code radi u pozadini? VS Code poziva g++ da iz .cpp datoteke napravi .exe, a zatim pokreće nastali program. Kasnije te korake može zapisati u konfiguracijske datoteke unutar mape .vscode.

12. Debugiranje programa

Debugiranje omogućuje da program zaustavite na odabranom retku, pregledate vrijednosti varijabli i izvršavate program korak po korak.

12.1. Zamijenite primjer programom za debugiranje

```
#include <iostream>

using namespace std;

int main()
{
    int a = 10;
    int b = 20;

    int suma = a + b;

    cout << "Suma = " << suma << endl;

    return 0;
}
```

12.2. Postavite breakpoint

Kliknite lijevo od retka:

```
int suma = a + b;
```

Pojavit će se crvena točka. To je breakpoint — debugger će zaustaviti program prije izvršavanja tog retka. Breakpoint se može uključiti/isključiti i tipkom F9.

12.3. Pokrenite debugger

Pritisnite F5. Ako VS Code prvi put traži konfiguraciju, odaberite “C++ (GDB/LLDB)” odnosno “C/C++: g++.exe build and debug active file” i pazite da se koristi g++.exe iz C:\msys64\ucrt64\bin. Build za debugiranje mora sadržavati opciju -g kako bi izvršna datoteka imala debug simbole.

AKO F5 NE POKRENE OČEKIVANI DEBUGGER

Otvorite Run and Debug → create a launch.json file → C++ (GDB/LLDB). U generiranoj konfiguraciji provjerite da je debugger GDB i da miDebuggerPath pokazuje na C:\msys64\ucrt64\bin\gdb.exe. Također provjerite da preLaunchTask kompajlira program s opcijom -g.

Očekivano ponašanje Program se kompajlira, pokreće pod GDB debuggerom i zaustavlja na breakpointu. U lijevom panelu Run and Debug pojavljuju se varijable i ostali debug podaci.

13. Praćenje varijabli i izvršavanje korak po korak

13.1. Pregledajte VARIABLES

Dok program stoji na breakpointu, u panelu VARIABLES vidjet ćete vrijednosti varijabli kao što su a i b. Nakon izvršavanja retka za zbroj pojavit će se i suma = 30.

13.2. Koristite Step Over i Step Into

Tipka	Naredba	Što radi
F5	Continue	nastavlja program do sljedećeg breakpointa ili kraja
F9	Breakpoint	uključuje/isključuje breakpoint na trenutnom retku
F10	Step Over	izvršava trenutni redak bez ulaska u pozvanu funkciju
F11	Step Into	ulazi u pozvanu funkciju i prati je redak po redak
Shift + F11	Step Out	izlazi iz trenutne funkcije
Shift + F5	Stop	prekida debugiranje

13.3. Dodajte WATCH izraze

U odjeljku WATCH možete dodati npr. a, b ili izraz a + b. VS Code će tijekom debugiranja prikazivati njihovu trenutnu vrijednost.

14. Preporučene compiler opcije

Za učenje, zadatke i posebno debugiranje preporučuje se kompajlirati uz upozorenja i debug informacije. Opcija -g je ključna za kvalitetno debugiranje na razini izvornog koda:

```
g++ -std=c++20 -Wall -Wextra -g main.cpp -o main.exe
```

Opcija	Značenje
-std=c++20	koristi C++20 standard
-Wall	uključuje važna upozorenja kompilera
-Wextra	uključuje dodatna korisna upozorenja
-g	ugrađuje debug informacije koje koristi GDB

Za nastavu Opcije -Wall i -Wextra posebno su korisne učenicima jer compiler upozorava na niz sumnjivih konstrukcija koje bi inače mogle ostati neprimijećene.

15. Najčešći problemi i rješenja

POSEBNO VAŽNO AKO JE PRIJE BIO INSTALIRAN DRUGI C++ ALAT

Ako where.exe g++ ili where.exe gdb kao prvi rezultat ne vraća C:\msys64\ucrt64\bin\..., provjerite redoslijed stavki u PATH-u. Stariji MinGW, Dev-C++, Qt ili sličan alat može uzrokovati da VS Code koristi pogrešan compiler ili debugger.

Problem	Što provjeriti / kako riješiti
"g++ is not recognized..."	PATH ne sadrži UCRT64 bin mapu ili je terminal otvoren prije promjene PATH-a. Provjerite C:\msys64\ucrt64\bin, zatvorite VS Code i ponovno ga pokrenite.
g++ radi u MSYS2, ali ne u VS Code-u	Najčešće je problem PATH. U novom VS Code terminalu pokrenite where.exe g++.

VS Code podcrtava <code>#include <iostream></code>	Otvorite Command Palette (Ctrl+Shift+P) → C/C++: Edit Configurations (UI) i kao Compiler path postavite C:\msys64\ucrt64\bin\g++.exe.
F5 ne pokreće debugger	Provjerite radi li gdb --version. Zatim ponovno odaberite C++/GDB konfiguraciju koja koristi UCRT64 alate.
Program se odmah zatvori	Pokrećite ga u integriranom VS Code terminalu ili pod debuggerom, gdje izlaz ostaje vidljiv.
Program se ne kompajlira	Pročitajte prvu compiler grešku u terminalu. Često uzrok bude zaboravljen ;, pogrešno napisano ime varijable ili nezatvorena zagrada.

16. Sažetak i brzi podsjetnik

Nakon jednokratne instalacije svakodnevni rad je jednostavan:

ZAVRŠNA PROVJERA ISPRAVNE INSTALACIJE

Prije nego učenik započne rad, provjerite: (1) gcc --version, g++ --version i gdb --version rade; (2) where.exe g++ i where.exe gdb kao prvi rezultat pokazuju C:\msys64\ucrt64\bin; (3) IntelliSense koristi C:\msys64\ucrt64\bin\g++.exe; (4) jednostavan program se može kompajlirati i pokrenuti; (5) F5 zaustavlja program na breakpointu i VARIABLES prikazuje vrijednosti varijabli.

- Otvorite mapu projekta u VS Code-u.
- Uredite ili napravite .cpp datoteku.
- Kompajlirajte/pokrenite program preko Run opcije ili terminala.
- Za debugiranje postavite breakpoint i pritisnite F5.
- Koristite F10/F11 za praćenje izvršavanja i VARIABLES/WATCH za vrijednosti.

Najvažnija kontrola instalacije U novom VS Code terminalu moraju raditi naredbe g++ --version i gdb --version. Ako rade, compiler i debugger su dostupni VS Code-u.

Brzi podsjetnik:

Ctrl + Shift + X	Extensions
Ctrl + `	Terminal
F5	Debug / Continue
F9	Breakpoint
F10	Step Over
F11	Step Into

Shift + F11
Shift + F5

Step Out
Stop

17. Službeni izvori i poveznice

Za ove upute korišteni su službeni izvori proizvođača alata. Budući da se nazivi verzija i izgled stranica mogu mijenjati, kod buduće instalacije uvijek je najbolje krenuti sa službenih poveznica ispod.

Visual Studio Code – Download

<https://code.visualstudio.com/Download>

Preuzimanje VS Code-a za Windows.

Visual Studio Code – C/C++ documentation

<https://code.visualstudio.com/docs/languages/cpp>

Microsoftov pregled C/C++ podrške u VS Code-u.

VS Code – Configure C++ with MinGW-w64

<https://code.visualstudio.com/docs/cpp/config-mingw>

Microsoftov vodič za GCC/GDB kroz MSYS2 na Windowsu.

C/C++ extension – Visual Studio Marketplace

<https://marketplace.visualstudio.com/items?itemName=ms-vscode.cpptools>

Službena Microsoftova C/C++ ekstenzija.

MSYS2 – službena stranica i instalacija

<https://www.msys2.org/>

Preuzimanje MSYS2 i osnovni instalacijski koraci.

MSYS2 – Environments

<https://www.msys2.org/docs/environments/>

Opis UCRT64 i drugih MSYS2 okruženja.

MSYS2 – UCRT64 toolchain paket

https://packages.msys2.org/groups/mingw-w64-ucrt-x86_64-toolchain

Popis paketa u UCRT64 toolchain grupi, uključujući GCC i GDB.

MSYS2 – Installer documentation

<https://www.msys2.org/docs/installer/>

Detalji o MSYS2 installeru.

Preporuka za nastavnika Ako ove upute dijelite učenicima, dobro je najprije zajedno odraditi poglavlja 2–7, a tek nakon provjere g++ i gdb nastaviti na prvi projekt i debugger.